Lean at MC2020

Release 0.1

Apurva Nakade
Jalex Stark

Nov 11, 2026

CONTENTS

1 Introduction 1

2 Logic in Lean - Part 1 3

3 Logic in Lean - Part 2 11

4 Infinitely Many Primes 19

5 Sqrt 2 is irrational 25

6 Bits & Pieces 29

7 Additional Problems for Practice 33

8 Pretty Symbols in Lean 39

9 Glossary of tactics 41

CHAPTER

ONE

INTRODUCTION

1.1 What is Lean?

Lean is an open source proof-checker and a proof-assistant. One can *explain* mathematical proofs to it and it can check their correctness. It also simplifies the proof writing process by providing `goals` and `tactics`.

Lean is built on top of a formal system called type theory. In type theory, the basic notions are "terms" and "types" — compare to "elements" and "sets" in set theory. Every term has a type, and types are just a special kind of term. Terms can be interpreted as mathematical objects, functions, propositions, or proofs. The only two things Lean can do are *create* terms and *check* their types. By iterating these two operations, we can teach Lean to verify complex mathematical proofs.

```
import Mathlib.Tactic

def x := 2 + 2                                    -- a natural number
def f (x : ℕ) := x + 3                            -- a function
def easy_theorem_statement := 2 + 2 = 4           -- a proposition
def fermats_last_theorem_statement                -- another proposition
    :=
  ∀ n : ℕ, n > 2 →
  ¬ (∃ x y z : ℕ, (x ^ n + y ^ n = z ^ n) ∧ (x ≠ 0) ∧ (y ≠ 0) ∧ (z ≠ 0))

theorem easy_proof : easy_theorem_statement := by  -- proof of easy_theorem
  rfl

theorem hard_proof : fermats_last_theorem_statement := by  -- cheating!
  sorry

#check x
#check f
#check easy_theorem_statement
#check fermats_last_theorem_statement
#check easy_proof
#check hard_proof
```

1.2 How to use these notes

Every once in a while, you will see a code snippet like this:

```
#eval "Hello, World!"
```

Clicking on the `try it!` button in the upper right corner will open a copy in a window so that you can edit it, and Lean provides feedback in the `Lean Infoview` window. We use this feature to provide exercises inline in the notes. We recommend attempting each exercise as you go along.

These notes are designed for a 5-day Lean crash course at Mathcamp 2020. On Days 1 and 2 you'll learn the basics of type theory and some basic `tactics` in Lean. On Days 3, 4, 5 you'll use these to prove increasingly complex theorems, namely the infinitude of primes and irrationality of $\sqrt{2}$.

These notes provide a sneak-peek into the world of theorem proving in Lean and are by no means comprehensive. It is recommended that you simultaneously attempt at least one of the following two options.

1. Play the Natural Number Game.
2. Read Theorem Proving in Lean 4.

The Natural Number Game is a fun (and highly addictive!) game that proves some basic properties of natural numbers in Lean. Theorem Proving in Lean 4 is a comprehensive online book that aims to cover all the theorem proving aspects of Lean in great detail. It is still under active development as of August 2020.

The Lean community is very welcoming to newcomers, and people are available on the Lean Zulip chat group round the clock to answer questions. You can also join Kevin Buzzard's Discord server which has a relatively younger crowd. You're highly encouraged to join one or both of these channels.

1.3 Acknowledgments

These notes are developed by Apurva Nakade and Jalex Stark with a lot of help from Mathcamp campers and Mathcamp staff Joanna and Maya (thanks!). Large chunks of these notes are taken from various learning resources available on the leanprover-community website.

1.4 Useful Links

1. Formalizing 100 theorems
2. Formalizing 100 theorems in Lean
3. **Articles, videos, blog posts, etc.**
    1. The Xena Project
    2. The Mechanization of Mathematics
    3. The Future of Mathematics
    4. Kevin Buzzard's Twitch channel. In particular, checkout this video about summer projects.
    5. Jalex Stark's Twitch channel.
4. Discord server
5. Lean Zulip chat group

CHAPTER

TWO

LOGIC IN LEAN - PART 1

Today's mission, should you choose to accept it, is to understand the philosophy of type theory (in Lean). Don't try to memorize anything, that will happen automatically. Instead, try to ~~realize that there is no spoon~~ do as many exercises as you can. Practice is the only way to learn a new programming language. And **always save your work**. The easiest way to do this is by bookmarking the Lean window in your web browser.

Lean is built on top of a logic system called *type theory*, which is an alternative to *set theory*. In type theory, instead of elements we have *terms* and every term has a *type*. When translated to math, terms can be either mathematical objects, functions, propositions, or proofs. The notation `x : X` stands for "`x` is a term of type `X`" or "`x` is an inhabitant of `X`". For the most part, you can think of a type as a set and terms as elements of the set.

2.1 Propositions as types

In set theory, a **proposition** is any statement that has the potential of being true or false, like `2 + 2 = 4`, `2 + 2 = 5`, "Fermat's last theorem", or "Riemann hypothesis". In type theory, there is a special type called `Prop` whose inhabitants are propositions. Furthermore, each proposition `P` is itself a type and the inhabitants of `P` are its proofs!

```
P : Prop     -- P is a proposition
hp : P       -- hp is a proof of P
```

As such, in type theory "producing a proof of `P`" is the same as "producing a term of type `P`" and so a proposition `P` is `true` if there exists a term `hp` of type `P`.

Notation. Throughout these notes, `P, Q, R, ...` will denote propositions.

2.1.1 Implication

In set theory, the proposition `P ⇒ Q` ("P implies Q") is true if either both `P` and `Q` are true or if `P` is false. In type theory, a proof of an implication `P ⇒ Q` is just a function `f : P → Q`. Given a function `f : P → Q`, every proof `hp : P` produces a proof `f(hp) : Q`. If `P` is false then `P` is *empty*, and there exists an empty function from an empty type to any type. Hence, in type theory we use `→` to denote implication.

2.1.2 Negation

In type theory, there is a special proposition `False : Prop` which has no proof (hence is *empty*). The negation of a proposition `¬ P` is the implication `P → False`. Such a function exists if and only if `P` itself is empty (empty function), hence `P → False` is inhabited if and only if `P` is empty which justifies using it as the definition of `¬ P`.

To summarize:

1. Proving a proposition `P` is equivalent to producing an inhabitant `hp : P`.
2. Proving an implication `P → Q` is equivalent to producing a function `f : P → Q`.
3. The negation, `¬ P`, is defined as the implication `P → False`.

2.1.3 Propositions in Lean

In Lean, a proposition and its proof are written using the following syntax.

```
import Mathlib.Tactic

theorem fermats_last_theorem
    (n : ℕ)
    (n_gt_2 : n > 2) :
    ¬ (∃ x y z : ℕ, (x ^ n + y ^ n = z ^ n) ∧ (x ≠ 0) ∧ (y ≠ 0) ∧ (z ≠ 0)) := by
  sorry
```

Let us parse the above statement. (Unlike the hypotheses and target, which Lean happily lets you spread across as many lines as you like, the proof itself is whitespace-sensitive: once you're inside a `by` block, each tactic goes on its own line, and how far a line is indented determines which proof (or sub-proof) it belongs to. We'll see why this matters once our proofs have more than one tactic in them.)

- `import Mathlib.Tactic` loads the mathlib4 library. It's what gives us notation like ℕ and every tactic we'll use, so most Lean files start with some `import` line — you can safely ignore it for now.
- `fermats_last_theorem` is the name of the theorem.
- `(n : ℕ)` and `(n_gt_2 : n > 2)` are the two *hypotheses*. The former says `n` is a natural number and the latter says that `n_gt_2` is a proof of `n > 2`.
- `:` is the delimiter between hypotheses and targets
- `¬ (∃ x y z : ℕ, (x^n + y^n = z^n) ∧ (x ≠ 0) ∧ (y ≠ 0) ∧ (z ≠ 0))` is the *target* of the theorem.
- `:= by ...` contains the proof. When you start your proof, Lean opens up a goal window for you to keep track of hypotheses and targets. **Your goal is to produce a term that has the type of the target**.

  ```
  -- example of Lean goal window
  n : ℕ -- hypothesis 1
  n_gt_2 : n > 2 -- hypothesis 2
  ⊢ ¬∃ x y z : ℕ, x ^ n + y ^ n = z ^ n ∧ x ≠ 0 ∧ y ≠ 0 ∧ z ≠ 0 -- target
  ```

- The commands you write after `by` are called *tactics*, one per line. `sorry` is an example of a tactic.

Even though they are not explicitly displayed, all the theorems in the Lean library are also hypotheses that you can use to close the goal.

2.2 Implications in Lean

We'll start learning tactics by proving implications in Lean. In the following sections, there are tables describing what a tactic does. Solve the following exercises to see the tactics in action.

The first two tactics we'll learn are `exact` and `intro`.

`exact`	If `P` is the target of the current goal and `hp` is a term of type `P`, then `exact hp` will close the goal. Mathematically, this saying "this is *exactly* what we were required to prove".
`intro`	If the target of the current goal is a function `P → Q`, then `intro hp` will produce a hypothesis `hp : P` and change the target to `Q`. Mathematically, this is saying that in order to define a function from `P` to `Q`, we first need to choose an arbitrary element of `P`.

```
-- ------------------------------------------------------------------------
--
-- ``exact``
--
--   If ``P`` is the target of the current goal and
--   ``hp`` is a term of type ``P``, then
--   ``exact hp`` will close the goal.
--
--
-- ``intro``
--
--   If the target of the current goal is a function ``P → Q``, then
--   ``intro hp`` will produce a hypothesis
--   ``hp : P`` and change the target to  ``Q``.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ------------------------------------------------------------------------

theorem tautology (P : Prop) (hp : P) : P := by
  sorry

theorem tautology' (P : Prop) : P → P := by
  sorry

example (P Q : Prop) : (P → (Q → P)) := by
  sorry

-- Can you find two different ways of proving the following?
example (P Q : Prop) : ((Q → P) → (Q → P)) := by
  sorry
```

The next two tactics are `have` and `apply`.

`have`	`have` is used to create intermediate variables. If `f` is a term of type `P → Q` and `hp` is a term of type `P`, then `have hq := f hp` creates the hypothesis `hq : Q`.
`apply`	`apply` is used for backward reasoning. If the target of the current goal is `Q` and `f` is a term of type `P → Q`, then `apply f` changes target to `P`. Mathematically, this is equivalent to saying "because `P` implies `Q`, to prove `Q` it suffices to prove `P`".

Often these two tactics can be used interchangeably. Think of `have` as reasoning forward and `apply` as reasoning backward. When writing a big proof, you often want a healthy combination of the two that makes the proof readable.

```
-- ------------------------------------------------------------------------
--
-- ``have``
--
--   If ``f`` is a term of type ``P → Q`` and
--   ``hp`` is a term of type ``P``, then
```

(continues on next page)

(continued from previous page)

```
--   ``have hq := f hp`` creates the hypothesis ``hq : Q`` .
--
--
-- ``apply``
--
--   If the target of the current goal is ``Q`` and
--   ``f`` is a term of type ``P → Q``, then
--   ``apply f`` changes target to ``P``.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ----------------------------------------------------------------------------

example (P Q R : Prop) (hp : P) (f : P → Q) (g : Q → R) : R := by
  sorry

example (P Q R S T U : Prop)
    (hpq : P → Q)
    (hqr : Q → R)
    (hqt : Q → T)
    (hst : S → T)
    (htu : T → U) :
    P → U := by
  sorry
```

For the following exercises, recall that `¬ P` is defined as `P → False`, `¬ (¬ P)` is `(P → False) → False`, and so on. Here are some hints if you get stuck.

```
-- ----------------------------------------------------------------------------
--
-- Recall that
--   ``¬ P`` is ``P → False``,
--   ``¬ (¬ P)`` is ``(P → False) → False``, and so on.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ----------------------------------------------------------------------------

theorem self_imp_not_not_self (P : Prop) : P → ¬ (¬ P) := by
  sorry

theorem contrapositive (P Q : Prop) : (P → Q) → (¬ Q → ¬ P) := by
  sorry

example (P : Prop) : ¬ (¬ (¬ P)) → ¬ P := by
  sorry
```

2.3 Proof by contradiction

You can prove exactly one of the converses of the above three using just `exact`, `intro`, `have`, and `apply`. Can you find which one?

```
-- ------------------------------------------------------------------------
--
-- You can prove exactly one of the following three using just
-- ``exact``, ``intro``, ``have``, and ``apply``.
--
-- Can you find which one?
--
-- ------------------------------------------------------------------------

theorem not_not_self_imp_self (P : Prop) : ¬ ¬ P → P := by
  sorry

theorem contrapositive_converse (P Q : Prop) : (¬ Q → ¬ P) → (P → Q) := by
  sorry

example (P : Prop) : ¬ P → ¬ ¬ ¬ P := by
  sorry
```

This is because it is not true that `¬ ¬ P = P` *by definition*, after all, `¬ ¬ P` is `(P → False) → False` which is drastically different from `P`. There is an extra axiom called **the law of excluded middle** which says that either `P` is inhabited or `¬ P` is inhabited (and there is no *middle* option) and so `P ↔ ¬ ¬ P`. This is the axiom that allows for proofs by contradiction. Lean provides us the following tactics to use it.

`exfalso`	Changes the target of the current goal to `False`. The name derives from "ex falso, quodlibet" which translates to "from contradiction, anything". You should use this tactic when there are contradictory hypotheses present.
`by_cases`	If `P : Prop`, then `by_cases P` creates two goals, the first with a hypothesis `hp : P` and second with a hypothesis `hp : ¬ P`. Mathematically, this is saying either `P` is true or `P` is false. `by_cases` is the most direct application of the law of excluded middle.
`by_contra`	If the target of the current goal is `Q`, then `by_contra` changes the target to `False` and adds `hnq : ¬ Q` as a hypothesis. Mathematically, this is proof by contradiction.
`push Not`	`push Not` simplifies negations in the target. For example, if the target of the current goal is `¬ ¬ P`, then `push Not` simplifies it to `P`. You can also push negations across a hypothesis `hp : P` using `push Not at hp`.
`contrapose!`	If the target of the current goal is `P → Q`, then `contrapose!` changes the target to `¬ Q → ¬ P`. If the target of the current goal is `Q` and one of the hypotheses is `hp : P`, then `contrapose! hp` changes the target to `¬ P` and changes the hypothesis to `hp : ¬ Q`. Mathematically, this is replacing the target by its contrapositive.

Even though the list is long, these tactics are almost all *obvious*. The only two slightly unusual tactics are `exfalso` and `by_cases`. You'll see `by_cases` in action later. For the following exercises, you only require `exfalso`, `push Not`, and `contrapose!`.

```
-- ------------------------------------------------------------------------
--
-- ``exfalso``
--
--   Changes the target of the current goal to ``False``.
--
-- ``push Not``
```

(continues on next page)

(continued from previous page)

```
--
--   ``push Not`` simplifies negations in the target.
--   You can push negations across a hypothesis ``hp : P`` using
--   ``push Not at hp``.
--
--
-- ``contrapose!``
--
--   If the target of the current goal is  ``P → Q``,
--   then ``contrapose!`` changes the target to  ``¬ Q → ¬ P``.
--
--   If the target of the current goal is ``Q`` and
--   one of the hypotheses is ``hp : P``, then
--   ``contrapose! hp`` changes the target to  ``¬ P`` and
--   changes the hypothesis to ``hp : ¬ Q``.
--
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ---------------------------------------------------------------------------

theorem not_not_self_imp_self (P : Prop) : ¬ ¬ P → P := by
  sorry

theorem contrapositive_converse (P Q : Prop) : (¬ Q → ¬ P) → (P → Q) := by
  sorry

example (P : Prop) : ¬ P → ¬ ¬ ¬ P := by
  sorry

theorem principle_of_explosion (P Q : Prop) : P → (¬ P → Q) := by
  sorry
```

2.4 Final Remarks

You might be wondering, if type theory is so cool why have I not heard of it before?

Many programming languages highly depend on type theory (that's where the term `datatype` comes from). Once you define a term `x : ℕ`, a computer can immediately check that all the manipulations you do with `x` are valid manipulations of natural numbers (so you don't accidentally divide by 0[1] , for example).

Unfortunately, this also means that the term `1 : ℕ` is different from the term `1 : ℤ`. In Lean, if you do `(1 : ℕ - 2 : ℕ)` you get `0 : ℕ` but if you do `(1 : ℤ - 2 : ℤ)` you get `-1 : ℤ`, that's because natural numbers and subtraction are not buddies. Another issue is that `1 : ℕ = 1 : ℤ` is not a valid statement in type theory. This is not the end of the world though. Lean allows you to *coerce* `1 : ℕ` to `1 : ℤ` if you want subtraction to work properly, or `1 : ℕ` to `1 : ℚ` if you want division to work properly.

This, and a few other such things, is what drives most mathematicians away from type theory. But these things are only difficult when you're first learning them. With practice, type theory becomes second nature, the same as set theory.

[1] Except under staff supervision.

footnotes

CHAPTER

THREE

LOGIC IN LEAN - PART 2

Your mission today is to wrap up the remaining bits of logic and move on to doing some "actual math". Remember to **always save your work**. You might find the *Glossary of tactics* page and the *Pretty symbols* page useful.

Before we move on to new stuff, let's understand what we did yesterday.

3.1 Behind the scenes

A note on brackets: It is not uncommon to compose half a dozen functions in Lean. The brackets get really messy and unwieldy. As such, Lean will often drop the brackets by following these conventions.

- The function `P → Q → R → S` stands for `P → (Q → (R → S))`.
- The expression `a + b + c + d` stands for `((a + b) + c) + d`.

An easy way to remember this is that arrows are bracketed on the right and binary operators on the left.

3.1.1 Proof irrelevance

It might feel a bit weird to say that a proposition has proofs as its inhabitants. Proofs can get huge and it seems unnecessary to have to remember not just the statement but also its proof. This is something we don't normally do in math. To hide this complication, in type theory there is an axiom, called *proof irrelevance*, which says that if `P : Prop` and `hp1 hp2 : P` then `hp1 = hp2`. Taking our *analogy* with sets further, you can think of a proposition as a set which is either empty or contains a single element (false or true). In fact, in some forms of type theory (e.g. homotopy type theory) this is taken as the definition of propositions. This is of course not true for general types. For example, `0 : ℕ ≠ 1 : ℕ`.

3.1.2 Proofs as functions

Every time you successfully construct a proof of a theorem, say

```
theorem tautology (P : Prop) : P → P := by
  intro hp
  exact hp
```

Lean constructs a *proof term* `tautology : ∀ P : Prop, P → P` (you can see this by typing `#check tautology`).

In type theory, the *for all* quantifier, ∀, is a generalized function, called a dependent function. For all practical purposes, we can think of `tautology` as having the type `(P : Prop) → (P → P)`. Note that this is not a function in the classical sense of the word because the codomain `(P → P)` *depends* on the input variable `P`. If `Q : Prop`, then `tautology Q` is a term of type `Q → Q`.

Consider a theorem with multiple hypotheses, say

```
theorem hello_world (hp : P) (hq : Q) (hr : R) : S
```

Once we provide a proof of it, Lean will create a proof term `hello_world : (hp : P) → (hq : Q) → (hr : R) → S`. So if we have terms `hp' : P`, `hq' : Q`, `hr' : R` then `hello_world hp' hq' hr'` (note the convenient lack of brackets) will be a term of type `S`.

Once constructed, any term can be used in a later proof. For example,

```
example (P Q : Prop) : (P → Q) → (P → Q) := by
  exact tautology (P → Q)
```

This is how Lean simulates mathematics. Every time you prove a theorem using tactics a *proof term* gets created. Because of proof irrelevance, Lean forgets the exact content of the proof and only remembers its type. All the proof terms can then be used in later proofs. All of this falls under the giant umbrella of the Curry–Howard correspondence.

We'll now continue our study of the remaining logical operators: *and* (∧), *or* (∨), *if and only if* (↔), *for all* (∀), *there exists* (∃).

3.2 And / Or

The operators *and* (∧) and *or* (∨) are very easy to use in Lean. Given a term `hpq : P ∧ Q`, there are tactics that let you create terms `hp : P` and `hq : Q`, and vice versa. Similarly for `P ∨ Q`, with a subtle change (see below).

Note that when multiple goals are open, you are trying to solve the topmost goal.

`obtain`	`obtain` is a general tactic that breaks a complicated term into simpler ones. If `hpq` is a term of type `P ∧ Q`, then `obtain ⟨hp, hq⟩ := hpq` breaks it into `hp : P` and `hq : Q`. If `fg` is a term of type `P ↔ Q`, then `obtain ⟨f, g⟩ := fg` breaks it into `f : P → Q` and `g : Q → P`. If `hpq` is a term of type `P ∨ Q`, then `obtain hp | hq := hpq` creates two goals and adds the hypothesis `hp : P` in one and `hq : Q` in the other.
`constructor`	`constructor` is a general tactic that breaks a complicated goal into simpler ones. If the target of the current goal is `P ∧ Q`, then `constructor` breaks up the goal into two goals with targets `P` and `Q`. If the target of the current goal is `P × Q`, then `constructor` breaks up the goal into two goals with targets `P` and `Q`. If the target of the current goal is `P ↔ Q`, then `constructor` breaks up the goal into two goals with targets `P → Q` and `Q → P`.
`left`	If the target of the current goal is `P ∨ Q`, then `left` changes the target to `P`.
`right`	If the target of the current goal is `P ∨ Q`, then `right` changes the target to `Q`.

```
-- ---------------------------------------------------------------------------
--
-- ``obtain``
--
--   ``obtain`` is a general tactic that breaks up complicated terms.
--   If ``hpq`` is a term of type ``P ∧ Q`` or ``P ↔ Q``, then use
--   ``obtain ⟨hp, hq⟩ := hpq``.
--   If ``hpq`` is a term of type ``P ∨ Q``, then use
--   ``obtain hp | hq := hpq``.
--
-- ``constructor``
--
--   If the target of the current goal is ``P ∧ Q`` or ``P ↔ Q``, then use
--   ``constructor``.
```

(continues on next page)

(continued from previous page)

```
--
-- ``left``/``right``
--
--   If the target of the current goal is ``P ∨ Q``, then use
--   either ``left`` or ``right`` (choose wisely).
--
-- ``exfalso``
--
--   Changes the target of the current goal to ``False``.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ---------------------------------------------------------------------------

example (P Q : Prop) : P ∧ Q → Q ∧ P := by
  sorry

example (P Q : Prop) : P ∨ Q → Q ∨ P := by
  sorry

example (P Q R : Prop) : P ∧ False ↔ False := by
  sorry

theorem principle_of_explosion (P Q : Prop) : P ∧ ¬ P → Q := by
  sorry
```

3.3 Quantifiers

As mentioned in the introduction, the *for all* quantifier, ∀, is a generalization of a function. As such the tactics for dealing with ∀ are the same as those for →.

`have`	If `hp` is a term of type `∀ x : X, P x` and `y` is a term of type `X` then `have hpy := hp y` creates a hypothesis `hpy : P y`.
`intro`	If the target of the current goal is `∀ x : X, P x`, then `intro x` creates a hypothesis `x : X` and changes the target to `P x`.

The *there exists* quantifier, ∃, in type theory is very intuitive. If you want to prove a statement `∃ x : X, P x` then you need to provide a witness. If you have a term `hp : ∃ x : X, P x` then from this you can extract a witness.

`obtain`	If `hp` is a term of type `∃ x : X, P x`, then `obtain ⟨x, key⟩ := hp` breaks it into `x : X` and `key : P x`.
`use`	If the target of the current goal is `∃ x : X, P x` and `y` is a term of type `X`, then `use y` changes the target to `P y` and tries to close the goal.

Finally, we know enough Lean tactics to start doing some fun stuff.

3.3.1 Barber paradox

Let's disprove the "barber paradox" due to Bertrand Russell. The claim is that in a certain town there is a (male) barber that shaves all the men who do not shave themselves. (Why is this a paradox?) Prove that this is a contradiction. Here are some hints if you get stuck.

```
-- ------------------------------------------------------------------------
--
-- ``by_cases``
--
--   If ``P`` is a proposition, then ``by_cases P`` creates two goals,
--     the first with a hypothesis ``hp : P`` and
--     second with a hypothesis ``hp : ¬ P``.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ------------------------------------------------------------------------

-- men is a type.
-- x : men means x is a man in the town
-- shaves x y is inhabited if x shaves y

variable (men : Type) (barber : men)
variable (shaves : men → men → Prop)

example : ¬ (∀ x : men, shaves barber x ↔ ¬ shaves x x) := by
  sorry
```

3.3.2 Mathcampers singing paradox

Assume that the main lounge is non-empty. At a fixed moment in time, there is someone in the lounge such that, if they are singing, then everyone in the lounge is singing. (See hints).

```
-- ------------------------------------------------------------------------
--
-- ``by_cases``
--
--   If ``P`` is a proposition, then ``by_cases P`` creates two goals,
--     the first with a hypothesis ``hp : P`` and
--     second with a hypothesis ``hp : ¬ P``.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ------------------------------------------------------------------------

-- camper is a type.
-- If x : camper then x is a camper in the main lounge.
-- singing x is inhabited if x is singing

theorem math_campers_singing_paradox
    (camper : Type)
    (singing : camper → Prop)
    (alice : camper) -- making sure that there is at least one camper in the lounge
    : ∃ x : camper, (singing x → (∀ y : camper, singing y)) := by
```

(continues on next page)

(continued from previous page)

```
  sorry
```

3.3.3 Relationship conundrum

A relation `r` on a type `X` is a map `r : X → X → Prop`. We say that `x` is *related* to `y` if `r x y` is inhabited.

- `r` is reflexive if `∀ x : X`, `x` is related to itself.
- `r` is symmetric if `∀ x y : X`, `x` is related to `y` implies `y` is related to `x`.
- `r` is transitive if `∀ x y z : X`, `x` is related to `y` and `y` is related to `z` implies `x` is related to `z`.
- `r` is connected if for all `x : X` there is a `y : X` such that `x` is related to `y`.

Show that if a relation is symmetric, transitive, and connected, then it is also reflexive.

```
import Mathlib.Tactic

variable (X : Type)

theorem reflexive_of_symmetric_transitive_and_connected
    (r : X → X → Prop)
    (h_symm : ∀ x y : X, r x y → r y x)
    (h_trans : ∀ x y z : X, r x y → r y z → r x z)
    (h_connected : ∀ x, ∃ y, r x y) :
    (∀ x : X, r x x) := by
  sorry
```

3.4 Proving "trivial" statements

In mathlib, divisibility for natural numbers is defined as the following *proposition*.

```
a ∣ b := (∃ k : ℕ, b = a * k)
```

For example, `2 ∣ 4` will be a proposition `∃ k : ℕ, 4 = 2 * k`. **Very important.** The statement `2 ∣ 4` is not saying that "2 divides 4 *is true*". It is simply a proposition that requires a proof.

Similarly, the mathlib library also contains a definition of `prime`. In mathlib, `Nat.Prime p` is defined as `Irreducible p`, a general notion of primality that makes sense in many number systems, not just ℕ. Fortunately, mathlib also proves that this is equivalent to the definition you are used to (see Nat.prime_def):

```
theorem Nat.prime_def {p : ℕ} :
  Nat.Prime p ↔
    2 ≤ p                                    -- p is at least 2
    ∧                                        -- and
    ∀ (m : ℕ), m ∣ p → m = 1 ∨ m = p          -- if m divides p, then m = 1 or m = p.
```

Same as with divisibility, for every natural number `n`, `Nat.Prime n` is a *proposition*. So that `Nat.Prime 101` requires a proof. It is possible to go down the rabbit hole and prove it using just the axioms of natural numbers. However, this might come at the cost of your sanity. Fortunately, there are tactics in Lean for proving trivial proofs such as these.

`norm_num`	`norm_num` is Lean's calculator. If the target has a proof that involves *only* numbers and arithmetic operations, then `norm_num` will close this goal. If `hp : P` is an assumption then `norm_num at hp` tries to use simplify `hp` using basic arithmetic operations.
`ring`	`ring` is Lean's symbolic manipulator. If the target has a proof that involves *only* algebraic operations, then `ring` will close the goal. If `hp : P` is an assumption then `ring at hp` tries to use simplify `hp` using basic algebraic operations.
`linarith`	`linarith` is Lean's inequality solver.
`simp`	`simp` is a very complex tactic that tries to use theorems from the mathlib library to close the goal. You should only ever use `simp` to *close a goal* because its behavior changes as more theorems get added to the library.

```
import Mathlib.Tactic

-- ---------------------------------------------------------------------------
--
-- ``norm_num``
--
--   Useful for arithmetic.
--
-- ``ring``
--
--   Useful for basic algebra.
--
-- ``linarith``
--
--   Useful for inequalities.
--
-- ``simp``
--
--   Complex simplifier. Use only to close goals.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ---------------------------------------------------------------------------

example : 1 > 0 := by
  sorry

example (m a b : ℕ) : m ^ 2 + (a + b) * m + a * b = (m + a) * (m + b) := by
  sorry

example : 101 ∣ 2020 := by
  sorry


#print Nat.Prime
example : Nat.Prime 101 := by
  sorry
```

(continues on next page)

(continued from previous page)

```
-- you will need the definition
-- a ∣ b := (∃ k : ℕ, b = a * k)
example (m a b : ℕ) : m + a ∣ m ^ 2 + (a + b) * m + a * b := by
  sorry

-- ``Nat.prime_def`` is a proof of ``Nat.Prime p ↔ (2 ≤ p ∧ ...)``.
-- Its first component ``Nat.prime_def.1`` is the ``→`` direction,
-- so try ``have h := Nat.prime_def.1 hp`` to get started
example (p : ℕ) (hp : Nat.Prime p) : ¬ (p = 1) := by
  sorry

-- if none of the simplifiers work, try doing ``contrapose!``
-- sometimes the simplifiers need a little help
example (n : ℕ) : 0 < n ↔ n ≠ 0 := by
  sorry
```

CHAPTER

FOUR

INFINITELY MANY PRIMES

Today we will prove that there are infinitely many primes using mathlib library. Our focus will be on how to *use* the library to prove more complicated theorems. Remember to always **save your work**.

4.1 Equality

So far we have not seen how to deal with propositions of the form `P = Q`, for example, `1 + 2 + ... + n = n(n + 1)/2`. Proving these propositions by hand requires messing around with the axioms of type theory. The standard trick is to make the LHS (almost) equal or to the RHS and then use one of the simplifiers (`norm_num`, `ring`, `linarith`, or `simp`) to close the goal. *Using* equalities on the other hand is very easy. The rewrite tactic (usually shortened to `rw`) lets you replace the left hand side of an equality with the right hand side.

`rw`	If `f` is a term of type `P = Q` (or `P ↔ Q`), then `rw [f]` searches for `P` in the target and replaces it with `Q`. `rw [← f]` searches for `Q` in the target and replaces it with `P`. Additionally, if `hr : R` is a hypothesis, then `rw [f] at hr` searches for `P` in the expression `R` and replaces it with `Q`. `rw [← f] at hr` searches for `Q` in the expression `R` and replaces it with `P`. Mathematically, this is saying "because `P = Q`, we can replace `P` with `Q` (or the other way around)".

To get the left arrow, type `\l` followed by tab.

```
import Mathlib.Tactic

-- ---------------------------------------------------------------------------
--
--   ``rw``
--
--     If ``f`` is a term of type ``P = Q`` (or ``P ↔ Q``), then
--     ``rw [f]`` replaces ``P`` with ``Q`` in the target.
--     Other variants:
--       ``rw [f] at hp``, ``rw [← f]``, ``rw [← f] at hr``.
--
--   Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ---------------------------------------------------------------------------

theorem add_self_self_eq_double
    (x : ℕ) :
    x + x = 2 * x := by
```

(continues on next page)

(continued from previous page)

```
  ring

-- For the following problem, use
--   mul_comm a b : a * b = b * a

example (a b c d : ℕ)
    (hyp : c = d * a + b)
    (hyp' : b = a * d) :
    c = 2 * (a * d) := by
  sorry

-- For the following problem, use
--   Nat.sub_self (x : ℕ) : x - x = 0

example (a b c d : ℕ)
    (hyp : c = b * a - d)
    (hyp' : d = a * b) :
    c = 0 := by
  sorry
```

4.1.1 Surjective functions

Recall that a function `f : X → Y` is surjective if for every `y : Y` there exists a term `x : X` such that `f(x) = y`. In type theory, for every function `f` we can define a corresponding proposition `surjective (f) := ∀ y, ∃ x, f x = y` and a function being surjective is equivalent to saying that the proposition `surjective(f)` is inhabited.

```
import Mathlib.Tactic
open Function

-- ---------------------------------------------------------------------
--
-- ``unfold``
--
--   If it gets hard to keep track of the definition of ``Surjective``,
--   you can use ``unfold Function.Surjective`` or ``unfold Function.Surjective at h``
--   to get rid of it.
--
-- Delete the ``sorry`` below and replace them with a legitimate proof.
--
-- ---------------------------------------------------------------------

variable (X Y Z : Type)
variable (f : X → Y) (g : Y → Z)

-- Surjective (f : X → Y) := ∀ y, ∃ x, f x = y

example
    (hf : Surjective f)
    (hg : Surjective g) :
    Surjective (g ∘ f) := by
  sorry
```

(continues on next page)

(continued from previous page)

```
example
    (hgf : Surjective (g ∘ f)) :
    Surjective g := by
  sorry
```

4.2 Creating subgoals

Often when we write a long proof in math, we break it up into simpler problems. This is done in Lean using the `have` tactic.

`have`	`have hp : P` creates a new goal with target `P` and adds `hp : P` as a hypothesis to the original goal.

The use of `have` that we have already seen is related to this one. When you use the tactic `have hq := f hp` Lean is internally replacing it with `have hq : Q := f hp`.

`have` is crucial for being able to use theorems from the library. To use these theorems you have to create terms that match the hypothesis *exactly*. Consider the following example. The type `n > 0` is not the same as `0 < n`. If you need a term of type `n > 0` and you only have `hn : 0 < n`, then you can use `have hn2 : n > 0 := by linarith` and you will have constructed a term `hn2` of type `n > 0`.

We will need the following lemma later. Remember to save your proof. (Here's a hint if you need one.) **Warning:** If you need to type the divisibility symbol, type `\mid`. This is **not** the vertical line on your keyboard.

```
import Mathlib.Tactic

-- ---------------------------------------------------------------------------
--
-- ``have``
--
--   ``have hp : P`` creates a new goal with target ``P`` and
--   adds ``hp : P`` as a hypothesis to the original goal.
--
-- You'll need the following theorem from the library:
--
-- Nat.dvd_sub' : k ∣ m → k ∣ n → k ∣ m - n
--
--    (Note that you don't need to provide m n k as inputs to dvd_sub'.
--    Lean can infer these from the rest of the expression.
--    More on this tomorrow.)
--
-- Delete the ``sorry`` below and replace it with a legitimate proof.
--
-- ---------------------------------------------------------------------------

theorem dvd_sub_one {p a : ℕ} : (p ∣ a) → (p ∣ a + 1) → (p ∣ 1) := by
  sorry
```

4.3 Infinitely many primes

We'll now prove that there are infinitely many primes. The strategy is to show that there is a prime greater than `n`, for every natural number `n`. We will choose this prime to be smallest non-trivial factor of `n! + 1`. We'll need the following definitions and theorems from the library.

Primes

- `m ∣ n := ∃ k : ℕ, n = m * k`
- `p.Prime := 2 ≤ p ∧ (∀ (m : ℕ), m ∣ p → m = 1 ∨ m = p)`
- `Nat.Prime.one_lt : p.Prime → 1 < p`

Factorials

- `n ! := Nat.factorial n`
- `Nat.factorial_pos : ∀ (n : ℕ), 0 < n !`
- `Nat.dvd_factorial : 0 < m → m ≤ n → m ∣ n !`

Smallest factor

- `Nat.minFac n :=` smallest non-trivial factor of `n`
- `Nat.minFac_prime : n ≠ 1 → (Nat.minFac n).Prime`
- `Nat.minFac_pos : ∀ (n : ℕ), 0 < Nat.minFac n`
- `Nat.minFac_dvd : ∀ (n : ℕ), Nat.minFac n ∣ n`

Check out Mathlib.Data.Nat.Prime.Basic for more theorems about primes. The exercise below is very open-ended. You should take your time, check the goal window at every step, and sketch out the proof on paper whenever you get lost.

```
import Mathlib.Tactic

theorem dvd_sub_one {p a : ℕ} : (p ∣ a) → (p ∣ a + 1) → (p ∣ 1) := by
  sorry

-- dvd_sub_one : (p ∣ a) → (p ∣ a + 1) → (p ∣ 1)
--
-- m ∣ n := ∃ k : ℕ, n = m * k
-- p.Prime :=  2 ≤ p ∧ (∀ (m : ℕ), m ∣ p → m = 1 ∨ m = p)
-- Nat.Prime.one_lt : p.Prime → 1 < p
--
-- n ! := Nat.factorial n
-- Nat.factorial_pos : ∀ (n : ℕ), 0 < n !
-- Nat.dvd_factorial : 0 < m → m ≤ n → m ∣ n !
--
-- Nat.minFac n := smallest non-trivial factor of n
-- Nat.minFac_prime : n ≠ 1 → (Nat.minFac n).Prime
-- Nat.minFac_pos : ∀ (n : ℕ), 0 < Nat.minFac n
-- Nat.minFac_dvd : ∀ (n : ℕ), Nat.minFac n ∣ n

theorem exists_infinite_primes (n : ℕ) : ∃ p, Nat.Prime p ∧ p ≥ n := by
  set p := Nat.minFac (Nat.factorial n + 1) with hp_def
  sorry
```

4.4 Final remarks

It would be great if there was a one-to-one correspondence between "hand-written proofs" and proofs in Lean. But that is far from the case. When we write proofs we leave out a lot of details without even realizing it and expect the reader to be intelligent enough to fill them in. This is both a bug and feature. On the one hand this makes proofs readable. On the other hand too many "obviously true" arguments make proofs undecipherable and often wrong.

Unlike human readers, computers are pretty dumb (as of writing these notes). They can only do what you tell them to do and you cannot expect them to "fill in the details". But it is humanly impossible to teach a computer every single trivial fact about, say the natural numbers. The Lean math library contains a lot of trivial theorems but this collection is far from comprehensive. So theorem proving in Lean often involves the following steps:

- Scan the library to see which definitions and theorems might be useful.
- Choose the right hypotheses and wording for your theorem to match the theorems in the library. (Sadly, changing the wording slightly might end up making the proof infinitely harder to prove.)
- Break the theorem into small lemmas so that you can use the simplifiers more frequently.

The hope is that one day we won't have to do this and a theorem proving AI will eliminate the difference between human proofs and machine proofs.

CHAPTER

FIVE

SQRT 2 IS IRRATIONAL

Today we will teach Lean that $\sqrt{2}$ is irrational. Let us start by reviewing some concepts we encountered yesterday.

5.1 Implicit arguments

Consider the following theorem which says that the smallest non-trivial factor of a natural number greater than 1 is a prime number.

```
Nat.minFac_prime : n ≠ 1 → (Nat.minFac n).Prime
```

It needs only one argument, namely a term of type `n ≠ 1`. But we have not told Lean what `n` is! That's because if we pass a term, say `hp : 2 ≠ 1` to `Nat.minFac_prime` then from `hp` Lean can infer that `n = 2`. `n` is called an *implicit* argument. An argument is made implicit by using curly brackets `{` and `}` instead of the usual `(` and `)` while defining the theorem.

```
theorem Nat.minFac_prime {n : ℕ} (hne1 : n ≠ 1) : (Nat.minFac n).Prime := ...
```

Sometimes the notation is ambiguous and Lean is unable to infer the implicit arguments. In such a case, you can force all the arguments to become explicit by putting an `@` symbol in front of the theorem. For example,

```
@Nat.minFac_prime : (n : ℕ) → n ≠ 1 → (Nat.minFac n).Prime
```

Use this sparingly as this makes the proof very hard to read and debug.

5.2 The two haves

We have seen two slightly different variants of the `have` tactic.

```
have hq := ...
have hq : ...
```

In the first case, we are defining `hq` to be the term on the right hand side. In the second case, we are saying that we do not know what the term `hq` is but we know it's type.

Let's consider the example of `Nat.minFac_prime` again. Suppose we want to conclude that the smallest factor of 10 is a prime. We will need a term of type `(Nat.minFac 10).Prime`. If this is the target, we can use `apply Nat.minFac_prime`. If not, we need a proof of `10 ≠ 1` to provide as input to `Nat.minFac_prime`. For this we'll use

```
have ten_ne_one : 10 ≠ 1
```

which will open up a goal with target `10 ≠ 1`. If on the other hand, you have another hypothesis, say `f : P → (10 ≠ 1)` and a term `hp : P`, then

```
have ten_ne_one := f hp
```

will immediately create a term of type `10 ≠ 1`. More generally, remember that

1. ":=" stands for definition. `x := ...` means that `x` is defined to be the right hand side.
2. ":" is a way of specifying type. `x : ...` means that the type of `x` is the right hand side.
3. "=" is only ever used in propositions and has nothing to do with terms or types.

5.3 Sqrt(2) is irrational

We will show that there do not exist non-zero natural numbers `m` and `n` such that

```
2 * m ^ 2 = n ^ 2  -- (*)
```

The crux of the proof is very easy. You simply have to start with the assumption that `m` and `n` are coprime *without any loss of generality* and derive a contradiction. But proving that *without loss of generality* is a valid argument requires quite a bit of effort. This proof is broken down into several parts. The first two parts prove `(*)` assuming that `m` and `n` are coprime. The rest of the parts prove the *without loss of generality* part.

For this problem, you'll need the following definitions.

- `m.gcd n : ℕ` is the gcd of `m` and `n`.
- `m.Coprime n` is defined to be the proposition `m.gcd n = 1`.

The descriptions of the library theorems you'll need are included as comments. Have fun!

5.3.1 Lemmas for proving (*) assuming m and n are coprime.

```
-- Nat.Prime.dvd_of_dvd_pow : p.Prime → p ∣ m ^ n → p ∣ m
lemma two_dvd_of_two_dvd_sq {k : ℕ}
    (hk : 2 ∣ k ^ 2) :
    2 ∣ k := by
  sorry

-- to switch the target from ``P = Q`` to ``Q = P``,
-- use the tactic ``symm``
lemma division_lemma_n {m n : ℕ}
    (hmn : 2 * m ^ 2 = n ^ 2) :
    2 ∣ n := by
  sorry

lemma div_2 {m n : ℕ} (hnm : 2 * m = 2 * n) : (m = n) := by
  linarith

lemma division_lemma_m {m n : ℕ}
    (hmn : 2 * m ^ 2 = n ^ 2) :
    2 ∣ m := by
  sorry
```

5.3.2 Prove (*) assuming m and n are coprime.

```
-- If ``1 < d``, ``d ∣ m``, and ``d ∣ n``, then ``d ∣ Nat.gcd m n`` (via
-- ``Nat.dvd_gcd``), and since ``m.Coprime n`` means ``Nat.gcd m n = 1``,
-- this contradicts ``1 < d``.

theorem sqrt2_irrational' :
    ¬ ∃ (m n : ℕ), 2 * m ^ 2 = n ^ 2 ∧ m.Coprime n := by
  rintro ⟨m, n, hmn, h_cop⟩
  -- rintro lets you destructure the hypothesis as you introduce it
  -- you get the brackets by typing ``\langle`` and ``\rangle``
  sorry
```

5.3.3 Lemmas for proving (*) assuming m ≠ 0

```
lemma ne_zero_ge_zero {n : ℕ}
    (hne : n ≠ 0) :
    (0 < n) := by
  contrapose! hne
  sorry

-- Nat.pow_pos : 0 < p → 0 < p ^ n  (n is inferred automatically)
lemma ge_zero_sq_ge_zero {n : ℕ} (hne : 0 < n) : (0 < n ^ 2) := by
  sorry

lemma cancellation_lemma {k m n : ℕ}
    (hk_pos : 0 < k ^ 2)
    (hmn : 2 * (m * k) ^ 2 = (n * k) ^ 2) :
    2 * m ^ 2 = n ^ 2 := by
  apply Nat.eq_of_mul_eq_mul_right hk_pos
  ring_nf
  ring_nf at hmn
  linarith [hmn]
```

5.3.4 Prove (*) assuming m ≠ 0

```
-- Nat.gcd_pos_of_pos_left : 0 < m → 0 < Nat.gcd m n
-- Nat.gcd_pos_of_pos_right : 0 < n → 0 < Nat.gcd m n
-- Nat.exists_coprime : ∀ (m n : ℕ), ∃ m' n', m'.Coprime n' ∧ m = m' * m.gcd n ∧ n = n
↪' * m.gcd n

theorem wlog_coprime :
    (∃ (m n : ℕ), 2 * m ^ 2 = n ^ 2 ∧ m ≠ 0) →
    (∃ (m' n' : ℕ), 2 * m' ^ 2 = n' ^ 2 ∧ m'.Coprime n') := by
  rintro ⟨m, n, hmn, hme0⟩
  set k := m.gcd n with hk
  -- might be useful to declutter
  -- you can replace all the ``m.gcd n`` with ``k`` using ``rw [← hk]`` if needed
  sorry

theorem sqrt2_irrational'' :
    ¬ ∃ (m n : ℕ), 2 * m ^ 2 = n ^ 2 ∧ m ≠ 0 := by
```

(continues on next page)

(continued from previous page)

```
  sorry
```

CHAPTER

SIX

BITS & PIECES

6.1 Namespaces

Lean provides us with the ability to group definitions into nested, hierarchical *namespaces*:

```
namespace vmcsp
  def tau := "TAU on M-Th from 1-3"
  #eval tau
end vmcsp

def tau := "no TAU on F"
#eval tau
#eval vmcsp.tau

open vmcsp

-- #eval tau  -- error: `tau` is now ambiguous between `_root_.tau` and `vmcsp.tau`
#eval vmcsp.tau
```

When we declare that we are working in the namespace `vmcsp`, every identifier we declare has a full name with prefix "`vmcsp`". Within the namespace, we can refer to identifiers by their shorter names, but once we end the namespace, we have to use the longer names.

The `open` command brings the shorter names into the current context. Often, when we import a theory file, we will want to open one or more of the namespaces it contains, to have access to the short identifiers. Further, if `x` is a term of type `Nat` and `f` is a term defined in namespace `Nat` then `Nat.f x` can be shortened to `x.f`. Note that $\mathbb{N}$ is just another notation for `Nat`.

6.2 Coercions

In type theory every term has a type and two terms of different types cannot be equal to each other. This makes it impossible to write statements like `|m|^2 = m^2` where `m : ` $\mathbb{Z}$ and `|m| : ` $\mathbb{N}$ is the absolute value of `m`. But in math, we do want this statement to be true! The roundabout way to deal with this is through *coercions*. Lean will coerce the above equality to live entirely in integers as $\uparrow$`|m|^2 = m^2`. This is done using an injective function $\mathbb{N} \to \mathbb{Z}$.

Sometimes it is possible (and necessary) to get rid of the coercions. For example, say we start out with $\uparrow$`|m|^2 = m^2` and eventually reduce it to $\uparrow$`|m|^2 = ` $\uparrow$`1`. The tactic for getting rid of coercions is `norm_cast` which will reduce the above expression to `|m|^2 = 1`.

`norm_cas`	`norm_cast` tries to clear out coercions. `norm_cast at hp` tries to clear out coercions at the hypothesis `hp`.

```
import Mathlib.Tactic

theorem sqrt2_irrational_nat :
    ¬ ∃ (m n : ℕ), 2 * (m * m) = (n * n) ∧ m ≠ 0 := by
  sorry

-- Assume the above theorem

lemma num_2 : (2 : ℚ).num = 2 := by
  norm_num

lemma den_2 : (2 : ℚ).den = 1 := by
  norm_num

-- q.den = denominator of q (valued in ℕ)
-- q.num = numerator of q (valued in ℤ)
--
-- for integer m,
-- m.natAbs = absolute value of m (valued in ℕ)
--
-- Rat.mul_self_den : ∀ (q : ℚ), (q * q).den = q.den * q.den
-- Rat.mul_self_num : ∀ (q : ℚ), (q * q).num = q.num * q.num
-- Int.natAbs_mul_self' : ∀ (a : ℤ), ↑(a.natAbs) * ↑(a.natAbs) = a * a
-- Rat.den_nz : ∀ (q : ℚ), q.den ≠ 0

-- Use ``simp at hp`` (or, interactively, ``simp? at hp``) to commute
-- products with coercions. See the goal window!

theorem sqrt2_irrational :
    ¬ (∃ q : ℚ, 2 = q * q) := by
  rintro ⟨q, key⟩
  have clear_denom := Rat.eq_iff_mul_eq_mul.mp key
  sorry
```

6.3 Type classes

Type classes are used to construct complex mathematical structures. Any family of types can be marked as a type class. We can then declare particular elements of a type class to be instances. You can think of a type class as "template" for constructing particular instances.

Consider the example of groups. A group is defined as a type class with the following attributes.

```
structure Group : Type u → Type u
fields:
Group.mul : {α : Type u} → [c : Group α] → α → α → α
Group.mul_assoc : ∀ {α : Type u} [c : Group α] (a b c_1 : α), a * b * c_1 = a * (b *␣
↪c_1)
Group.one : {α : Type u} → [c : Group α] → α
Group.one_mul : ∀ {α : Type u} [c : Group α] (a : α), 1 * a = a
Group.mul_one : ∀ {α : Type u} [c : Group α] (a : α), a * 1 = a
Group.inv : {α : Type u} → [c : Group α] → α → α
Group.inv_mul_cancel : ∀ {α : Type u} [c : Group α] (a : α), a⁻¹ * a = 1
```

If you look at the source code you'll see that `class Group` is built gradually by extending multiple classes (the real hierarchy in mathlib4 has a few more intermediate steps than shown here, to support both additive and multiplicative notation, but the idea is the same).

```
class One  (α : Type u) where
  one : α
-- a group has an identity element

class Mul  (α : Type u) where
  mul : α → α → α
-- a group has multiplication

class Inv  (α : Type u) where
  inv : α → α
-- a group has an inverse function

class Semigroup (G : Type u) extends Mul G where
  mul_assoc : ∀ a b c : G, a * b * c = a * (b * c)
-- the multiplication is associative

class Monoid (M : Type u) extends Semigroup M, One M where
  one_mul : ∀ a : M, 1 * a = a
  mul_one : ∀ a : M, a * 1 = a
-- multiplication by one is trivial

class Group (α : Type u) extends Monoid α, Inv α where
  inv_mul_cancel : ∀ a : α, a⁻¹ * a = 1
-- every element has an inverse
```

To define an arbitrary group `G` we first create it as a type `G : Type` and then assume it is a group using `[Group G]`. You can also prove that existing types are instances of `Group` using the `instance` keyword. Type classes allow us to prove theorems in great generality. For example, any theorem about groups can immediately be applied to integers once we show that integers are an instance of `Group`. If you look at Mathlib.Algebra.Group.Int you'll see the code that proves $\mathbb{Z}$ is an instance of several type classes.

```
import Mathlib.GroupTheory.OrderOfElement
import Mathlib.Tactic

#print Group

class CyclicGroup (G : Type*) extends Group G where
  has_generator : ∃ g : G, ∀ x : G, ∃ n : ℤ, x = g ^ n

-- zpow_add : ∀ {G : Type u_1} [inst : Group G] (a : G) (m n : ℤ), a ^ (m + n) = a ^␣
↪m * a ^ n

lemma mul_comm_of_cyclic
    {G : Type*}
    [hc : CyclicGroup G]
    (g : G) :
    ∀ a b : G, a * b = b * a := by
  have has_generator := hc.has_generator
  sorry
```

6.4 Final remarks

That's a wrap! Over the last five days you went from wrangling `P → Q` with `intro` and `exact` to building type classes and proving that there are infinitely many primes and that $\sqrt{2}$ is irrational. That is genuinely research-level formalization, and you did it in a week.

The tactics you practiced — `intro`, `have`, `obtain`, `rw`, `norm_num`, and friends — are exactly the tactics used to formalize serious mathematics in mathlib, a rapidly growing, community-maintained library that by now covers a huge swath of undergraduate and graduate mathematics.

If you want to keep going:

1. Finish the Natural Number Game if you haven't already.
2. Work through Theorem Proving in Lean 4 for a more systematic treatment of the theory behind what we did.
3. Pick a theorem you like and try to formalize it. 100 theorems in Lean is a good source of inspiration, and the mathlib4 repository is a good place to look for the building blocks you'll need.
4. Stick around on the Lean Zulip chat group — people there are very welcoming to newcomers, and it's the best way to find out what the community is working on.

Thanks for spending the week with us, and good luck with your future proofs!

CHAPTER

SEVEN

ADDITIONAL PROBLEMS FOR PRACTICE

The five days above are the whole course, but here are a few extra problems that never made it into the schedule. They're independent of each other and of the daily material, so pick whichever looks fun. Full worked solutions are linked in the sidebar under "Solutions on GitHub".

7.1 Induction & recursion practice

7.1.1 A rewriting puzzle

Let $p : \mathbb{N} \to \text{Prop}$, and suppose $p(n)$ holds for some fixed n. Suppose further that $p(m) \iff p(m+8)$ and $p(m+3) \iff p(m)$ for every $m \in \mathbb{N}$. Show that $p(n+1)$ holds.

```
import Mathlib.Tactic

example
    (p : ℕ → Prop) (n : ℕ) (hn : p n)
    (h8 : ∀ n, p n ↔ p (n + 8))
    (h3 : ∀ n, p (n + 3) ↔ p n) :
    p (n + 1) := by
  sorry
```

7.1.2 Two small types

Show that the type with no elements is not the type with one element, i.e. $\text{Fin}\,0 \neq \text{Fin}\,1$.

```
import Mathlib.Tactic

example : Fin 0 ≠ Fin 1 := by
  sorry
```

7.1.3 Reflexivity from symmetry and transitivity

Let r be a relation on $\mathbb{N}$ that is symmetric and transitive, and suppose every $x \in \mathbb{N}$ is r-related to *some* y (i.e. r is *serial*). Show that r must in fact be reflexive.

```
import Mathlib.Tactic

theorem reflexive_of_symmetric_and_transitive (r : ℕ → ℕ → Prop)
    (h_symm : Std.Symm r) (h_trans : IsTrans ℕ r)
    (h_connected : ∀ x, ∃ y, r x y) :
```

(continues on next page)

(continued from previous page)

```
    Std.Refl r where
  refl x := by
    sorry
```

7.1.4 Every number is even or odd

Show that every $n \in \mathbb{N}$ can be written as $2k$ or $2k + 1$ for some k.

```
import Mathlib.Tactic

lemma even_or_odd (n : ℕ) :
    (∃ k, n = 2 * k) ∨ ∃ k, n = 2 * k + 1 := by
  sorry
```

7.1.5 Induction in both directions

Let $p : \mathbb{Z} \to$ Prop satisfy $p(n) \Rightarrow p(n + 1)$ and $p(n) \Rightarrow p(n - 1)$ for every $n \in \mathbb{Z}$. Show that p holds everywhere on $\mathbb{Z}$ if and only if it holds at 0. (Ordinary induction on $\mathbb{N}$ only walks in one direction — here you have to walk outward from 0 in both directions.)

```
import Mathlib.Tactic

example
    (p : ℤ → Prop)
    (p_succ : ∀ n, p n → p (n + 1))
    (p_pred : ∀ n, p n → p (n - 1)) :
    (∀ n, p n) ↔ p 0 := by
  sorry
```

7.1.6 Gauss's formula

Define $f : \mathbb{N} \to \mathbb{N}$ recursively by $f(0) = 0$ and $f(n + 1) = (n + 1) + f(n)$, so that $f(n) = 0 + 1 + \cdots + n$. Show $f(1) = 1$ and, more generally, $2f(n) = n(n + 1)$ for every n.

```
import Mathlib.Tactic

-- landing in ℕ recursion (rather than subtraction) avoids the perils of nat␣
↪subtraction
def f : ℕ → ℕ
  | 0 => 0
  | (n + 1) => n + 1 + f n

example : f 1 = 1 := by
  sorry

example (n : ℕ) : 2 * f n = n * (n + 1) := by
  sorry
```

7.1.7 Telescoping sums

Let R be a commutative ring, $a \in R$, and $n \in \mathbb{N}$. Show

$$(1-a)\sum_{k=0}^{n-1} a^k = 1 - a^n.$$

Prove it twice: once by finding the matching lemma already in mathlib, and once "by hand" by induction on n.

```
import Mathlib.Tactic

variable {R : Type*} [CommRing R]

-- find the matching lemma in mathlib
example (n : ℕ) (a : R) :
    (1 - a) * ∑ k ∈ Finset.range n, a ^ k = 1 - a ^ n := by
  sorry

-- now redo it "by hand", by induction on n
example (n : ℕ) (a : R) :
    (1 - a) * ∑ k ∈ Finset.range n, a ^ k = 1 - a ^ n := by
  sorry
```

7.2 Primes & parity practice

7.2.1 Triple negation

For any proposition P, show $\neg\neg\neg P \to \neg P$.

```
import Mathlib.Tactic

example (P : Prop) : ¬ ¬ ¬ P → ¬ P := by
  sorry
```

7.2.2 Primes are two or odd

Show that every prime p satisfies $p = 2$ or p is odd. (There is a lemma in mathlib that says exactly this — see if you can find it.)

```
import Mathlib.Tactic

example (p : ℕ) : p.Prime → p = 2 ∨ p % 2 = 1 := by
  sorry
```

7.2.3 Consecutive primes and their sum

Call two primes $p < q$ *consecutive* if no prime lies strictly between them. Work up to showing: if $p < q$ are consecutive primes, neither equal to 2, then $p + q$ factors as a product of *three* numbers each greater than 1.

Build the proof in stages:

(a) An even prime must equal 2.

(b) A sum of two odd numbers is even.

(c) If $b < bc$, then $c > 1$.

(d) A composite number $k \geq 2$ splits as $k = ab$ for some $1 < a, b < k$.

(e) Put it together: since $p, q \neq 2$ are prime, they're both odd, so $p + q = 2k$ is even for some k strictly between p and q; since k can't be prime, it splits as $k = bc$ with $b, c > 1$, giving $p + q = 2 \cdot b \cdot c$.

```
import Mathlib.Tactic

-- (a) an even prime must be 2
lemma eq_two_of_even_prime {p : ℕ} (hp : p.Prime) (h_even : Even p) : p = 2 := by
  sorry

-- (b) a sum of two odd numbers is even
lemma even_of_odd_add_odd
    {a b : ℕ} (ha : ¬ Even a) (hb : ¬ Even b) :
    Even (a + b) := by
  sorry

-- (c) if b < b * c, then c is at least 2
lemma one_lt_of_nontrivial_factor
    {b c : ℕ} (hb : b < b * c) :
    1 < c := by
  sorry

-- (d) a composite k ≥ 2 splits as a product of two smaller factors, each > 1
lemma nontrivial_product_of_not_prime
    {k : ℕ} (hk : ¬ k.Prime) (two_le_k : 2 ≤ k) :
    ∃ a b, a < k ∧ b < k ∧ 1 < a ∧ 1 < b ∧ a * b = k := by
  sorry

-- (e) the main theorem
theorem three_fac_of_sum_consecutive_primes
    {p q : ℕ} (hp : p.Prime) (hq : q.Prime) (hpq : p < q)
    (p_ne_2 : p ≠ 2) (q_ne_2 : q ≠ 2)
    (consecutive : ∀ k, p < k → k < q → ¬ k.Prime) :
    ∃ a b c, p + q = a * b * c ∧ a > 1 ∧ b > 1 ∧ c > 1 := by
  sorry
```

7.3 The frog problem

This one is the first problem from Mathcamp's 2012 qualifying quiz, formalized.

A *frog* lives on the lily pads numbered $0, 1, 2, \ldots$ (the natural numbers). It starts on pad 0 at time 0, and it has some fixed step size $s \in \mathbb{N}$: every second, it jumps s pads to the right.

$$\text{location}(0) = 0 \qquad \text{location}(n+1) = \text{location}(n) + s$$

For every step size s, `frogOfStepSize s` below is *the* frog that takes steps of size s. Work through the following:

(a) Show that a frog with step size s is at position $n \cdot s$ at time n.

(b) Every frog is *the* frog of its step size (a frog is determined entirely by its step size).

(c) **The quiz problem.** You get to check exactly one lily pad each second, without knowing the frog's step size in advance. Show that there is a strategy (a choice of which pad to check at each time n) that is guaranteed to

eventually catch the frog — i.e. to check the very pad the frog is on at that moment — no matter what step size the frog turns out to have.

```
import Mathlib.Tactic

@[ext]
structure Frog where
  -- A frog hangs out on the natural number line of lily pads
  location : ℕ → ℕ
  -- At time 0, it sits on location 0
  location_zero : location 0 = 0
  -- For some fixed step size,
  step_size : ℕ
  -- the frog jumps `step_size` units to the right each second
  step : ∀ n, location (n + 1) = location n + step_size

-- (a)
lemma frog_explicit_formula (f : Frog) :
    ∀ n, f.location n = n * f.step_size := by
  sorry

-- the frog that takes steps of size `step_size`
def frogOfStepSize (step_size : ℕ) : Frog where
  location := fun n => n * step_size
  location_zero := by simp
  step_size := step_size
  step := by intro n; ring

-- (b)
lemma frog_eq_frog_of_step_size (f : Frog) :
    f = frogOfStepSize f.step_size := by
  sorry

-- (c) the quiz problem
lemma catch_the_frog :
    ∃ (strategy : ℕ → ℕ),
    -- no matter how fast the frog travels,
    ∀ step_size,
    -- you'll eventually catch it
    ∃ catch_time > 0,
    strategy catch_time = (frogOfStepSize step_size).location catch_time := by
  sorry
```

CHAPTER

EIGHT

PRETTY SYMBOLS IN LEAN

To produce a pretty symbol in Lean, type the *editor shortcut* followed by space or tab.

Unicode	Editor Shortcut	Definition
$\to$	`\to`	function or implies
$\leftrightarrow$	`\iff`	if and only if
$\leftarrow$	`\l`	used by the `rw` tactic
$\neg$	`\not`	negation operator
$\wedge$	`\and`	and operator
$\vee$	`\or`	or operator
$\exists$	`\exists`	there exists quantifier
$\forall$	`\forall`	for all quantifier
$\mid$	`\mid`	divisibility[1]
$\mathbb{N}$	`\nat`	type of natural numbers
$\mathbb{Z}$	`\int`	type of integers
$\circ$	`\circ`	composition of functions
$\neq$	`\ne`	not equal to

[1] Be very careful! The symbol for divisibility is not the | symbol on your keyboard. Lean will through a cryptic error if you use it.

CHAPTER

NINE

GLOSSARY OF TACTICS

9.1 Implications in Lean

`exact`	If `P` is the target of the current goal and `hp` is a term of type `P`, then `exact hp` will close the goal. Mathematically, this saying "this is *exactly* what we were required to prove".
`intro`	If the target of the current goal is a function `P → Q`, then `intro hp` will produce a hypothesis `hp : P` and change the target to `Q`. Mathematically, this is saying that in order to define a function from `P` to `Q`, we first need to choose an arbitrary element of `P`.

`have`	`have` is used to create intermediate variables. If `f` is a term of type `P → Q` and `hp` is a term of type `P`, then `have hq := f hp` creates the hypothesis `hq : Q`.
`apply`	`apply` is used for backward reasoning. If the target of the current goal is `Q` and `f` is a term of type `P → Q`, then `apply f` changes target to `P`. Mathematically, this is equivalent to saying "because `P` implies `Q`, to prove `Q` it suffices to prove `P`".

9.2 Proof by contradiction

`exfalso`	Changes the target of the current goal to `False`. The name derives from "ex falso, quodlibet" which translates to "from contradiction, anything". You should use this tactic when there are contradictory hypotheses present.
`by_cases`	If `P : Prop`, then `by_cases P` creates two goals, the first with a hypothesis `hp : P` and second with a hypothesis `hp : ¬ P`. Mathematically, this is saying either `P` is true or `P` is false. `by_cases` is the most direct application of the law of excluded middle.
`by_contra`	If the target of the current goal is `Q`, then `by_contra` changes the target to `False` and adds `hnq : ¬ Q` as a hypothesis. Mathematically, this is proof by contradiction.
`push Not`	`push Not` simplifies negations in the target. For example, if the target of the current goal is `¬ ¬ P`, then `push Not` simplifies it to `P`. You can also push negations across a hypothesis `hp : P` using `push Not at hp`.
`contrapo`	If the target of the current goal is `P → Q`, then `contrapose!` changes the target to `¬ Q → ¬ P`. If the target of the current goal is `Q` and one of the hypotheses is `hp : P`, then `contrapose! hp` changes the target to `¬ P` and changes the hypothesis to `hp : ¬ Q`. Mathematically, this is replacing the target by its contrapositive.

9.3 And / Or

`obtain`	`obtain` is a general tactic that breaks a complicated term into simpler ones. If `hpq` is a term of type `P ∧ Q`, then `obtain ⟨hp, hq⟩ := hpq` breaks it into `hp : P` and `hq : Q`. If `fg` is a term of type `P ↔ Q`, then `obtain ⟨f, g⟩ := fg` breaks it into `f : P → Q` and `g : Q → P`. If `hpq` is a term of type `P ∨ Q`, then `obtain hp | hq := hpq` creates two goals and adds the hypothesis `hp : P` in one and `hq : Q` in the other.
`constructor`	`constructor` is a general tactic that breaks a complicated goal into simpler ones. If the target of the current goal is `P ∧ Q`, then `constructor` breaks up the goal into two goals with targets `P` and `Q`. If the target of the current goal is `P × Q`, then `constructor` breaks up the goal into two goals with targets `P` and `Q`. If the target of the current goal is `P ↔ Q`, then `constructor` breaks up the goal into two goals with targets `P → Q` and `Q → P`.
`left`	If the target of the current goal is `P ∨ Q`, then `left` changes the target to `P`.
`right`	If the target of the current goal is `P ∨ Q`, then `right` changes the target to `Q`.

9.4 Quantifiers

`have`	If `hp` is a term of type `∀ x : X, P x` and `y` is a term of type `X` then `have hpy := hp y` creates a hypothesis `hpy : P y`.
`intro`	If the target of the current goal is `∀ x : X, P x`, then `intro x` creates a hypothesis `x : X` and changes the target to `P x`.

`obtain`	If `hp` is a term of type `∃ x : X, P x`, then `obtain ⟨x, key⟩ := hp` breaks it into `x : X` and `key : P x`.
`use`	If the target of the current goal is `∃ x : X, P x` and `y` is a term of type `X`, then `use y` changes the target to `P y` and tries to close the goal.

9.5 Proving "trivial" statements

`norm_num`	`norm_num` is Lean's calculator. If the target has a proof that involves *only* numbers and arithmetic operations, then `norm_num` will close this goal. If `hp : P` is an assumption then `norm_num at hp` tries to simplify `hp` using basic arithmetic operations.
`ring`	`ring` is Lean's symbolic manipulator. If the target has a proof that involves *only* algebraic operations, then `ring` will close the goal. If `hp : P` is an assumption then `ring at hp` tries to simplify `hp` using basic algebraic operations.
`linarith`	`linarith` is Lean's inequality solver.
`simp`	`simp` is a very complex tactic that tries to use theorems from the mathlib library to close the goal. You should only ever use `simp` to *close a goal* because its behavior changes as more theorems get added to the library.

9.6 Equality

`rw`	If `f` is a term of type `P = Q` (or `P ↔ Q`), then `rw [f]` searches for `P` in the target and replaces it with `Q`. `rw [← f]` searches for `Q` in the target and replaces it with `P`. If additionally, `hr : R` is a hypothesis, then `rw [f] at hr` searches for `P` in the expression `R` and replaces it with `Q`. `rw [← f] at hr` searches for `Q` in the expression `R` and replaces it with `P`. Mathematically, this is saying because `P = Q`, we can replace `P` with `Q` (or the other way around).
`norm_cast`	`norm_cast` tries to clear out coercions between types (e.g. $\mathbb{N} \to \mathbb{Z}$ or $\mathbb{N} \to \mathbb{Q}$). `norm_cast at hp` tries to clear out coercions in the hypothesis `hp`.